

MILESTONE 0

El Plano.

Arquitectura del software antes de la primera línea de código.

HOY
21 abr 2026

ENTREGA M0
28 abr 2026

TP · RIR-API

¿Dónde estamos?

El TP se desarrolla en cuatro entregas incrementales. La primera vence en una semana.

MILESTONE	ENTREGAN	FECHA	PESO
M0 El Plano	Arquitectura · repo · issues	28 Abr — próxima clase	5%
M1 Generación	Ruido rosa · sine sweep	19 May	15%
M2 Procesamiento	Filtros · deconvolución · RI	16 Jun	20%
M3 Producto	API REST · informe · demo	7 Jul	30%

Importante: el 28/4 no solo entregan M0 — también lo presentan en vivo al docente.

El plano viene antes que la obra.

Nadie construye un edificio sin un plano. Con software pasa lo mismo: arrancar a codear sin plan termina en un proyecto que **colapsa en M2**.

M0 no es un milestone de código. Es el acuerdo del grupo:

- ¿Qué estamos construyendo?
- ¿Cómo se organizan las partes?
- ¿Quién hace qué?
- ¿En qué orden?

5% de la nota. 100% de la base del resto del TP.

SEÑALES DE UN PLANO FLOJO

- Nadie sabe qué carpeta toca
- Tres personas tocan el mismo archivo
- Issues vagos: "hacer M1"
- Para M2, el código es un caos

Una API REST.

Un servidor que recibe pedidos HTTP y devuelve datos en JSON.

Ábranla ahora en el navegador:

<https://rir-api.onrender.com/docs>

O probenla desde la terminal:

```
# request
curl https://rir-api.onrender.com/health

# response (JSON)
{
  "status": "healthy",
  "version": "0.1.0",
  "timestamp": "2026-04-21T15:30:00"
}
```

ENDPOINT

Una URL que hace una cosa. Ej: `GET /health` devuelve "estoy vivo".

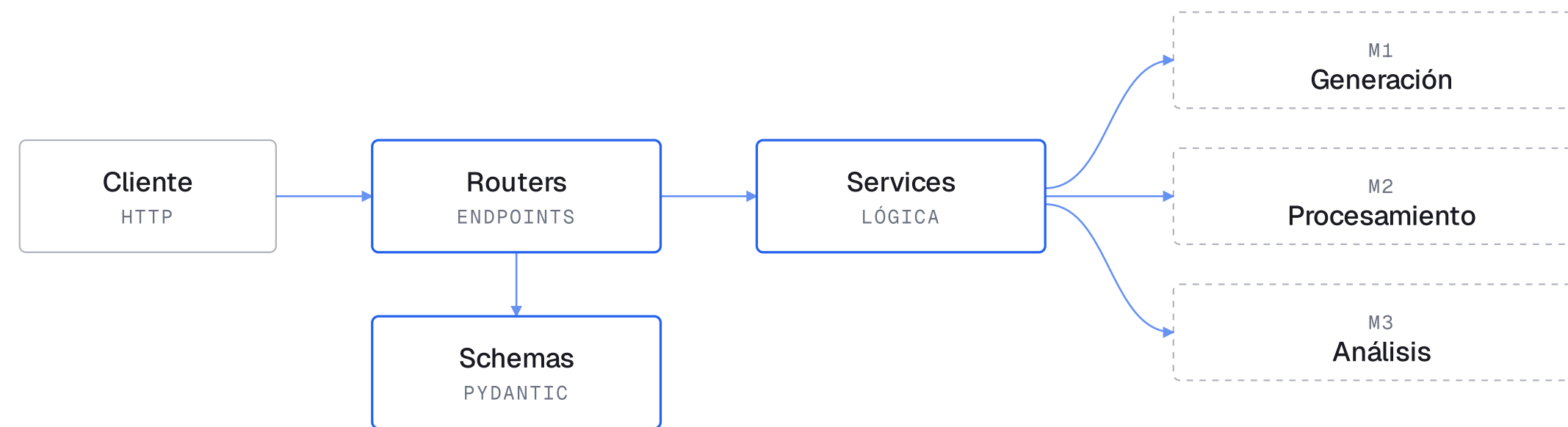
REQUEST · RESPONSE

El cliente manda audio y parámetros; la API responde con T60, C80, etc.

JSON

Formato para pasar datos. `{"t60": 1.2, "c80": 5.4}`

Tres capas. Memorícenlo.



ROUTERS · puerta de entrada

```
@router.post("/pink-noise")
async def pink_noise(req: PinkNoiseReq):
    audio = services.generar_ruido_rosa(
        req.duracion, req.fs
    )
    return {"audio": audio.tolist()}
```

Reciben el request, llaman al service, devuelven JSON. No calculan.

SERVICES · cerebro

```
def generar_ruido_rosa(
    duracion: float, fs: int
) -> np.ndarray:
    n = int(duracion * fs)
    # algoritmo Voss-McCartney
    return signal
```

Funciones puras. No saben de HTTP ni de JSON.

SCHEMAS · aduana

```
class PinkNoiseReq(BaseModel):
    duracion: float = Field(gt=0, le=60)
    fs: int = Field(default=44100)
```

Validan que los datos entren y salgan con la forma correcta.

README.md

Lo primero que mira el docente cuando abre el repo.

DEBE TENER

- Nombre y descripción (1-2 párrafos)
- **Integrantes con rol asignado**
señales · procesamiento · testing/CI · docs
- Instrucciones que funcionen copy-paste
- Estructura del proyecto comentada
- Branching strategy

BUENO

```
git clone https://...  
cd rir-api  
uv sync  
uv run uvicorn app.main:app --reload
```

FLOJO

"Para instalar, pip install (ver requirements.txt)"

Diagrama de arquitectura.

Formato: Mermaid (en el README) o draw.io (PNG).

DEBE MOSTRAR

- Las 3 capas (routers / services / schemas)
- Módulos de **M1, M2 y M3** — no solo M0
- Flujo de datos: request → validación → service → response
- Dependencias externas relevantes

Mermaid = diagramas en texto. Se versionan en Git. Probalo en mermaid.live.

SINTAXIS MÍNIMA

```
```\nmermaid\ngraph LR\n    Client --> R[Routers]\n    R --> S[Services]\n    S --> Gen[Generación]\n    S --> Proc[Procesamiento]\n    S --> An[Análisis]\n    R --> Sch[Schemas]\n```\n
```

# GitHub Issues.

Al menos 10 issues, cada uno con título, descripción, labels y asignación.

## MALO

"Hacer M1"

Demasiado grande. Nadie sabe qué entra ni cuándo está listo.

## BUENO

"Implementar `generar_ruido_rosa` con Voss-McCartney"

### Criterios:

- Recibe `(duracion, fs)`, devuelve `np.ndarray`
- Espectro cae ~3 dB/octava
- Test unitario que lo verifique

`milestone-1` `complejidad-media` · @maria

Regla de oro: un issue = algo que una persona puede terminar en una sesión de trabajo.



# Estructura del repo.

Buena noticia: ya está hecha. Forkean el template, no lo reescriben.

```
rir-api/
├─ pyproject.toml ← dependencias
├─ README.md
├─ app/
│ ├─ main.py ← entrada FastAPI
│ ├─ settings.py
│ ├─ routers/
│ │ └─ health.py ← ya funciona
│ ├─ schemas/
│ └─ services/
├─ tests/
│ └─ test_placeholder.py
├─ docs/
└─ data/
```

## QUÉ HACER

1. Forkear `template_repo`

2. Clonar localmente

3. Levantar el server:

```
uv sync
```

```
uv run uvicorn app.main:app --reload
```

4. Abrir `localhost:8000/docs`

5. Correr tests: `pytest -v`

Si todo esto funciona, cumplieron los requisitos técnicos de M0.

# El endpoint `/health` .

El más tonto posible: devuelve "estoy vivo". Confirma que el servidor corre.

```
from datetime import datetime
from fastapi import APIRouter

router = APIRouter()

@router.get("/health")
async def health_check():
 """Verifica que la API esta funcionando correctamente."""
 return {
 "status": "healthy",
 "version": "0.1.0",
 "timestamp": datetime.now().isoformat(),
 }
```

Criterio de MO: que esto responda `200 OK` cuando levantan el server.

# Las reglas mínimas.

## BRANCHING

- `main` protegida — merge solo vía **pull request**
- `feature/nombre-descriptivo` — una rama por issue
- Commits: recomendado [Conventional Commits](#)  
`feat:` · `fix:` · `docs:` · `test:`

## NO OLVIDAR

Agregar a los docentes como **colaboradores del repo**.

Settings → Collaborators → Add people.

Sin esto no podemos ver ni corregir el TP.

# Checklist de aprobación.

El 28 de abril, esto tiene que estar tildado.

- ☐ Repositorio accesible por los docentes
- ☐ README completo con instrucciones que funcionan
- ☐ Diagrama con todos los módulos de M1, M2 y M3
- ☐ Al menos 10 issues con labels y asignaciones
- ☐ Estructura que levanta con `uvicorn`
- ☐ Endpoint `/health` responde `200 OK`
- ☐ Al menos un test que pase con `pytest`
- ☐ Branching strategy documentada

Ocho checks. Nada más, nada menos.



# Para llevar a casa.

## DEL TP

- [API de referencia — Swagger](#)
- [API de referencia — ReDoc](#)
- Especificación:  
`trabajo_practico/especificacion/m0_arquitectura.md`
- Template: `trabajo_practico/template_repo/`

## HERRAMIENTAS

- [FastAPI](#)
- [uv — gestor de paquetes](#)
- [Mermaid Live Editor](#)
- [Conventional Commits](#)
- [GitHub Issues — buenas prácticas](#)

PRÓXIMOS PASOS

# Esta semana, planifiquen.

Próxima clase: presentan M0 en vivo.